SEGUNDO EXAMEN PARCIAL

TRABAJO EN GRUPO

El siguiente examen será realizado y expuesto en grupos de dos/tres estudiantes. Deberá ser presentado en algunas de las fechas designadas en el cronograma de clases.

ENUNCIADO

Elegir una temática de acuerdo con los intereses del grupo y realizar un proyecto Full-Stack con las tecnologías vistas en clase. El proyecto deberá estar dividido en dos repositorios diferentes: *backend* y *frontend*.

Repositorio de *backend*

1. Debe ser una API REST estructurada bajo las buenas prácticas recomendadas según las tecnologías utilizadas.
2. Debe tener persistencia a través de una base de datos que soporte el modelo orientado a documentos, con al menos cuatro colecciones.
3. Debe permitir las operaciones **CRUD (Create, Read, Update, Delete)** completas y con sus respectivas validaciones para al menos una entidad del modelo. Las demás operaciones quedan sujetas a los casos de uso decididos por el grupo.
4. Debe manejar correctamente las excepciones en las operaciones críticas.
5. Debe contar con al menos dos tipos de usuarios: **Usuario Final** y **Usuario Administrador**. El grupo deberá determinar los permisos y restricciones de cada rol sobre los casos de uso brindados por la API y su correcta implementación.
6. Debe gestionar el manejo de sesión y autenticación de los usuarios mediante JWT.
7. Debe estar correctamente desplegada en un servicio de hosting que la soporte.
8. Debe poseer una documentación que permita que cualquier desarrollador pueda levantar el servidor, entender su funcionamiento y conocer cómo interactuar con él mediante la API:
    - Modelo de datos: Esquematizar cada una de las colecciones de la base de datos (nombre, descripción), describiendo a su vez, para cada campo, su nombre, tipo de dato, validaciones y reglas.
    - Archivo **README.md** con la configuración del entorno (software necesario para correr el proyecto, pasos de instalación, variables de entorno, modos de ejecución)
    - Descripción de cada uno de los *endpoints* (ruta, método HTTP, descripción, parámetros, *request body* (si aplica), ejemplo de respuesta exitosa, ejemplos de errores posibles)
    - Estructura del proyecto: Un resumen del árbol de carpetas del proyecto explicando la responsabilidad de cada una.

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0

Repositorio de *frontend*

1. Debe ser una SPA (*Single Page Application*) *responsive*, adaptándose correctamente a distintos tamaños de pantalla (móvil y escritorio)
2. Debe proporcionar retroalimentación visual al usuario durante las operaciones asíncronas.
3. Debe conectarse e interactuar exclusivamente con el repositorio del *backend* para obtener, crear, modificar y eliminar datos.
4. Debe implementar validaciones básicas en los *forms* antes de enviar datos al servidor.
5. Debe gestionar el manejo de sesión y autenticación de los usuarios mediante JWT.
6. Debe estar correctamente desplegada en un servicio de hosting que la soporte.
7. Debe poseer una documentación que permita que cualquier desarrollador pueda levantar el servidor, entender su funcionamiento y conocer cómo interactuar con él mediante la API:
    - Archivo **README.md** con la configuración del entorno (software necesario para correr el proyecto, pasos de instalación, variables de entorno, modos de ejecución)
    - Estructura del proyecto: Un resumen del árbol de carpetas del proyecto explicando la responsabilidad de cada una.
    - Descripción de cada una de las librerías utilizadas y su propósito.

FORMATO DE ENTREGA

Subir el código fuente a un repositorio de GitHub y compartir el enlace a través del medio que indique el docente. El repositorio debe ser público o bien privado, pero con permiso de acceso al docente para poder efectuar la corrección.

El formato de entrega es EXCLUYENTE. No se aceptan archivos por mail, Google Drive u otros.

Cada repositorio debe contener el código fuente correspondiente y la documentación pertinente. En particular, el repositorio de *backend* debe poseer el/los archivos que permitan regenerar la base de datos de forma local (si es que no se trabaja con base de datos en la nube).

DEFENSA

La defensa del proyecto constará de dos etapas, a resolverse el mismo día del examen:

Etapa 1: Defensa oral grupal

Las y los estudiantes que conformen el grupo de trabajo deberán exponer su proyecto el día del examen ante el docente y sus compañeros, demostrando su correcto funcionamiento e indicando cómo han participado en su construcción.

Si se aprueba esta etapa, pasarán a la Etapa 2, de lo contrario, deberán recuperar en la próxima fecha.

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0

Etapa 2: Defensa escrita individual

En el contexto actual, donde el acceso a herramientas de inteligencia artificial, tutoriales en línea y recursos automatizados para el desarrollo de software es cada vez más amplio, se vuelve indispensable no solo demostrar que un proyecto funciona, sino también comprender profundamente su construcción. Durante esta etapa, cada estudiante deberá completar un examen escrito, donde deberá demostrar que puede relacionar los conceptos de la programación funcional vistos durante la cursada con su aplicación en el proyecto realizado.

Si se aprueba esta etapa, queda aprobado el segundo parcial, de lo contrario, el estudiante deberá recuperar solo la Etapa 2 en la próxima fecha.

CRITERIO DE APROBACIÓN

- Para que el examen sea aprobado con nota igual o superior a **8 puntos (Aprobación Directa)**, deberán estar resueltos correctamente **todos** los puntos de **ambos** repositorios (*frontend* y *backend*)
- Para que el examen sea aprobado con nota **entre 6 y 7 puntos (Aprobación No Directa)**, deberán estar resueltos correctamente **todos** los puntos del **r**epositorio de *backend*.

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0